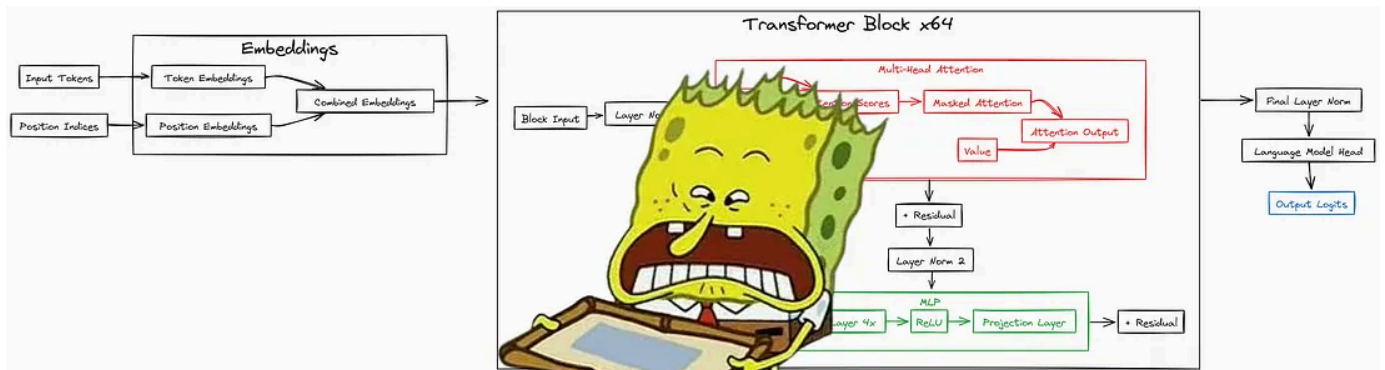


Level Up Coding · [Follow publication](#)


✦ Member-only story

Building a 2 Billion Parameter LLM from Scratch Using Python

It starts making sense

26 min read · Jan 15, 2025



Fareed Khan

Follow



Listen



Share



More

 Read this story for free: [link](#)

Quick Note — We will train a 2 billion-parameter LLM starting from scratch using The Pile dataset. As a result, we get an LLM that outputs perfect grammar and punctuation in responses, with shorter contexts making sense, but not the entire response.

I am looking for a PhD position in AI. You can take a look at my [resume](#) or [GitHub profile](#).

Previously, I wrote an article on Medium about creating a [2.3+ million-parameter LLM using the Tiny Shakespeare dataset](#), but the output didn't make sense.

2.3 Million Parameter LLM Output

ZELBETH:

 Sey solmenter! tis tonguerered **if**

Vurint as steolated have loven OId the queend refore

Are been, good plmp:

Proforne, wiftes swleen, was no blunderesd a a quain beath!

Tybell is my gateer stalk smend as be matious dazest

I had a thought, what if I make the transformer architecture smaller and less complex, and the training data more diverse? Then, how big of a model could a single person, using their nearly dead GPU, create in terms of parameters that can speak proper grammar and generate text that makes some sense?

This is the output of our trained model following this blog:

I found that 13+ million-parameter are enough to start making sense in terms of proper grammar and punctuation, which is a positive point. This means we can use a very specific dataset to further fine-tune our previously trained model for a narrowed task. We might end up with a model under 1 billion parameters or even around 500 million parameters that is perfect for our specific use case, especially for running it on private data securely.

I recommend you **first train a 13+ million-parameter** model using the script available in my GitHub repository. You will get results within one day, instead of waiting for a longer time, or if your local GPU might not be strong enough to train a billion-parameter model.

GitHub Code Overview

All the code is available in my GitHub repository:

GitHub - FareedKhan-dev/train-llm-from-scratch

Contribute to FareedKhan-dev/train-llm-from-scratch development by creating an account on GitHub.

github.com

The codebase is organized as follows:

```
train-llm-from-scratch/  
├── src/  
│   ├── models/  
│   │   ├── mlp.py # Definition of the Multi-Layer Perceptron (MLP) module  
│   │   ├── attention.py # Definitions for attention mechanisms (single-head, multi-head)  
│   │   ├── transformer_block.py # Definition of a single Transformer block  
│   │   └── transformer.py # Definition of the main Transformer model  
│   ├── config/  
│   │   └── config.py # Contains default configurations (model parameters, file paths, etc.)  
│   ├── data_loader/  
│   │   └── data_loader.py # Contains functions for creating data loaders/iterators  
│   └── scripts/  
│       ├── train_transformer.py # Script for training the Transformer model  
│       ├── data_download.py # Script for downloading the dataset  
│       ├── data_preprocess.py # Script for preprocessing the downloaded data  
│       └── generate_text.py # Script for generating text using a trained model  
├── data/ # Directory to store the dataset  
├── train/ # Contains training data  
│   └── val/ # Contains validation data  
└── models/ # Directory where trained models are saved
```

- The `scripts/` directory includes scripts for tasks such as dataset downloading, data preprocessing, model training, and text generation using the trained model.
- The `src/models/` directory houses the implementation of key components, including the transformer model, multi-layer perceptron (MLP), attention mechanisms, and transformer blocks.
- The `config/` directory contains configuration files specifying the default parameters for the project.
- The `data_loader/` directory provides functions for creating data loaders and iterators.

Table of Contents

1. [Prerequisites and Training Time](#)
2. [Installing Module](#)
3. [Importing Libraries](#)
4. [Preparing the Training Data](#)
5. [Transformer Overview](#)
6. [Multi Layer Perceptron \(MLP\)](#)
7. [Single Head Attention](#)
8. [Multi Head Attention](#)
9. [Transformer Block](#)
10. [The Final Model](#)
11. [Batch Processing](#)
12. [Training Parameters](#)
13. [Training the Model](#)
14. [Saving the Trained Model](#)
15. [Training Loss](#)
16. [Generating Text](#)
17. [What's Next](#)

Prerequisites and Training Time

Make sure you have a basic understanding of object-oriented programming (OOP) and neural networks (NN). Familiarity with PyTorch will also be helpful for coding.

You will need a GPU to train your model. Colab or Kaggle T4 will work for training a 13+ million-parameter model, but they will fail for billion-parameter training. Take a look at the comparison:

Installing Module

Make sure Git is installed in your environment. You first need to clone the repository:

```
git clone https://github.com/FareedKhan-dev/train-llm-from-scratch.git
cd train-llm-from-scratch
```

Then you can install the required dependencies:

```
pip install -r requirements.txt
```

Importing Libraries

Let's import the required libraries that will be used throughout this blog:

```

# PyTorch for deep learning functions and tensors
import torch
import torch.nn as nn
import torch.nn.functional as F

# Numerical operations and arrays handling
import numpy as np

# Handling HDF5 files
import h5py

# Operating system and file management
import os

# Command-line argument parsing
import argparse

# HTTP requests and interactions
import requests

# Progress bar for loops
from tqdm import tqdm

# JSON handling
import json

# Zstandard compression library
import zstandard as zstd

# Tokenization library for large language models
import tiktoken

# Math operations (used for advanced math functions)
import math

```

Preparing the Training Data

Our training dataset needs to be diverse, containing information from different domains, and The Pile is the right choice for it. Although it is 825 GB in size, we will stick to only a small portion of it, i.e., 5%–10%. Let's first download the dataset and see how it works. I will be downloading the version available on [HuggingFace](https://huggingface.co/datasets/monology/pile-uncopyrighted).

```

# Download validation dataset
!wget https://huggingface.co/datasets/monology/pile-uncopyrighted/resolve/main/val.jsonl.zst

# Download the first part of the training dataset
!wget https://huggingface.co/datasets/monology/pile-uncopyrighted/resolve/main/train/00.jsonl.zst

# Download the second part of the training dataset
!wget https://huggingface.co/datasets/monology/pile-uncopyrighted/resolve/main/train/01.jsonl.zst

# Download the third part of the training dataset
!wget https://huggingface.co/datasets/monology/pile-uncopyrighted/resolve/main/train/02.jsonl.zst

```

It will take some time to download, but you can also limit the training dataset to just one file, `00.jsonl.zst`, instead of three. It is already split into train/val/test. Once it's done, make sure to place the files correctly in their respective directories.

```

import os
import shutil
import glob

# Define directory structure
train_dir = "data/train"
val_dir = "data/val"

# Create directories if they don't exist
os.makedirs(train_dir, exist_ok=True)
os.makedirs(val_dir, exist_ok=True)

```

```
# Move all train files (e.g., 00.jsonl.zst, 01.jsonl.zst, ...)
train_files = glob.glob("*.jsonl.zst")
for file in train_files:
    if file.startswith("val"):
        # Move validation file
        dest = os.path.join(val_dir, file)
    else:
        # Move training file
        dest = os.path.join(train_dir, file)
    shutil.move(file, dest)
```

Our dataset is in the `.jsonl.zst` format, which is a compressed file format commonly used for storing large datasets. It combines JSON Lines (`.jsonl`), where each line represents a valid JSON object, with Zstandard (`.zst`) compression. Let's read a sample of one of the downloaded files and see how it looks.

```
in_file = "data/val/val.jsonl.zst" # Path to our validation file

with zstd.open(in_file, 'r') as in_f:
    for i, line in tqdm(enumerate(in_f)): # Read first 5 lines
        data = json.loads(line)
        print(f"Line {i}: {data}") # Print the raw data for inspection
        if i == 2:
            break

#### OUTPUT ####
Line: 0
{
  "text": "Effect of sleep quality ... epilepsy.",
  "meta": {
    "pile_set_name": "PubMed Abstracts"
  }
}

Line: 1
{
  "text": "LLMops a new GitHub Repository ...",
  "meta": {
    "pile_set_name": "Github"
  }
}
```

Now we need to encode (tokenize) our dataset. Our goal is to have an LLM that can at least output proper words. For that, we need to use an already available tokenizer. We will use the `tiktoken` open-source tokenizer by OpenAI. We will use the `r50k_base` tokenizer, which is used for the ChatGPT (GPT-3) model, to tokenize our dataset.

We need to create a function for this to avoid duplication, as we will be tokenizing both the train and validation datasets.

```
def process_files(input_dir, output_file):
    """
    Process all .zst files in the specified input directory and save encoded tokens to an HDF5 file.

    Args:
        input_dir (str): Directory containing input .zst files.
        output_file (str): Path to the output HDF5 file.
    """
    with h5py.File(output_file, 'w') as out_f:
        # Create an expandable dataset named 'tokens' in the HDF5 file
        dataset = out_f.create_dataset('tokens', (0,), maxshape=(None,), dtype='i')
        start_index = 0

        # Iterate through all .zst files in the input directory
        for filename in sorted(os.listdir(input_dir)):
            if filename.endswith(".jsonl.zst"):
                in_file = os.path.join(input_dir, filename)
                print(f"Processing: {in_file}")

                # Open the .zst file for reading
```

```

with zstd.open(in_file, 'r') as in_f:
    # Iterate through each line in the compressed file
    for line in tqdm(in_f, desc=f"Processing {filename}"):
        # Load the line as JSON
        data = json.loads(line)

        # Append the end-of-text token to the text and encode it
        text = data['text'] + "<|endoftext|>"
        encoded = enc.encode(text, allowed_special={'<|endoftext|>'})
        encoded_len = len(encoded)

        # Calculate the end index for the new tokens
        end_index = start_index + encoded_len

        # Expand the dataset size and store the encoded tokens
        dataset.resize(dataset.shape[0] + encoded_len, axis=0)
        dataset[start_index:end_index] = encoded

        # Update the start index for the next batch of tokens
        start_index = end_index

```

There are two important points regarding this function:

1. We are storing the tokenized data in an HDF5 file, which allows us flexibility for quicker data access while training the model.
2. Appending the <|endoftext|> token marks the end of each text sequence, signaling to the model that it has reached the end of a meaningful context, which helps in generating coherent outputs.

Now we can simply encode our train and validation datasets using:

```

# Define tokenized data output directories
out_train_file = "data/train/pile_train.h5"
out_val_file = "data/val/pile_dev.h5"

# Loading tokenizer of (GPT-3/GPT-2 Model)
enc = tiktoken.get_encoding('r50k_base')

# Process training data
process_files(train_dir, out_train_file)

# Process validation data
process_files(val_dir, out_val_file)

```

Let's take a look at the sample of our tokenized data:

```

with h5py.File(out_val_file, 'r') as file:
    # Access the 'tokens' dataset
    tokens_dataset = file['tokens']

    # Print the dtype of the dataset
    print(f"Dtype of 'tokens' dataset: {tokens_dataset.dtype}")

    # load and print the first few elements of the dataset
    print("First few elements of the 'tokens' dataset:")
    print(tokens_dataset[:10]) # First 10 token

#### OUTPUT ####
Dtype of 'tokens' dataset: int32

First few elements of the 'tokens' dataset:
[ 2725  6557   83 23105  157  119  229  77 5846 2429]

```

We have prepared our dataset for training. Now we will code the transformer architecture and look into its theory correspondingly.

Transformer Overview

Let's have a quick look at how a transformer architecture is used to process and understand text. It works by breaking text into smaller pieces called tokens and predicting the next token in the sequence. A transformer has many layers, called transformer blocks, stacked on top of each other, with a final layer at the end to make the prediction.

Each transformer block has two main components:

- **Self-Attention Heads:** These figure out which parts of the input are most important for the model to focus on. For example, when processing a sentence, the attention heads can highlight relationships between words, such as how a pronoun relates to the noun it refers to.
- **MLP (Multi-Layer Perceptron):** This is a simple feed-forward neural network. It takes the information emphasized by the attention heads and processes it further. The MLP has an input layer that receives data from the attention heads, a hidden layer that adds complexity to the processing, and an output layer that passes the results to the next transformer block.

Together, the attention heads act as the “what to think about” part, while the MLP is the “how to think about it” part. Stacking many transformer blocks allows the model to understand complex patterns and relationships in the text, but this is not always guaranteed.

Instead of looking at the original paper diagram, let's visualize a simpler and easier architecture diagram that we will be coding.

Transformer Architecture by [Fareed Khan](#)

Let's read through the flow of our architecture that we will be coding:

1. Input tokens are converted to embeddings and combined with position information.
2. The model has 64 identical transformer blocks that process data sequentially.
3. Each block first runs multi-head attention to look at relationships between tokens.
4. Each block then processes data through an MLP that expands and then compresses the data.
5. Each step uses residual connections (shortcuts) to help information flow.
6. Layer normalization is used throughout to stabilize training.
7. The attention mechanism calculates which tokens should pay attention to each other.
8. The MLP expands the data to 4x size, applies ReLU, and then compresses it back down.
9. The model uses 16 attention heads to capture different types of relationships.
10. The final layer converts the processed data into vocabulary-sized predictions.
11. The model generates text by repeatedly predicting the next most likely token.

Multi Layer Perceptron (MLP)

MLP is a fundamental building block within the transformer's feed-forward network. Its role is to introduce non-linearity and learn complex relationships within the embedded representations. When defining an MLP module, an important parameter is `n_embed`, which defines the dimensionality of the input embedding.

The MLP typically consists of a hidden linear layer that expands the input dimension by a factor (often 4, which we will use), followed by a non-linear activation function, commonly ReLU. This structure allows our network to learn more complex features. Finally, a projection linear layer maps the expanded representation back to the original embedding dimension. This sequence of transformations enables the MLP to refine the representations learned by the attention mechanism.

MLP by Fareed Khan

```
# --- MLP (Multi-Layer Perceptron) Class ---

class MLP(nn.Module):
    """
    A simple Multi-Layer Perceptron with one hidden layer.

    This module is used within the Transformer block for feed-forward processing.
    It expands the input embedding size, applies a ReLU activation, and then projects it back
    to the original embedding size.
    """
    def __init__(self, n_embed):
        super().__init__()
        self.hidden = nn.Linear(n_embed, 4 * n_embed) # Linear layer to expand embedding size
        self.relu = nn.ReLU() # ReLU activation function
        self.proj = nn.Linear(4 * n_embed, n_embed) # Linear layer to project back to original size

    def forward(self, x):
        """
        Forward pass through the MLP.

        Args:
            x (torch.Tensor): Input tensor of shape (B, T, C), where B is batch size,
                               T is sequence length, and C is embedding size.

        Returns:
            torch.Tensor: Output tensor of the same shape as the input.
        """
        x = self.forward_embedding(x)
        x = self.project_embedding(x)
        return x

    def forward_embedding(self, x):
        """
        Applies the hidden linear layer followed by ReLU activation.

        Args:
            x (torch.Tensor): Input tensor.

        Returns:
            torch.Tensor: Output after the hidden layer and ReLU.
        """
        x = self.relu(self.hidden(x))
        return x

    def project_embedding(self, x):
        """
        Applies the projection linear layer.
```

```

    Args:
        x (torch.Tensor): Input tensor.

    Returns:
        torch.Tensor: Output after the projection layer.
    """
    x = self.proj(x)
    return x

```

We just coded our MLP part, where the `__init__` method initializes a hidden linear layer that expands the input embedding size (`n_embed`) and a projection layer that reduces it back. ReLU activation is applied after the hidden layer. The `forward` method defines the data flow through these layers, applying the hidden layer and ReLU via `forward_embedding`, and the projection layer via `project_embedding`.

Single Head Attention

The attention head is the core part of our model. Its purpose is to focus on relevant parts of the input sequence. When defining a Head module, some important parameters are `head_size`, `n_embed`, and `context_length`. The `head_size` parameter determines the dimensionality of the key, query, and value projections, influencing the representational capacity of the attention mechanism.

The input embedding dimension `n_embed` defines the size of the input to these projection layers. `context_length` is used to create a causal mask, ensuring that the model only attends to preceding tokens.

Within the Head, linear layers (`nn.Linear`) for key, query, and value are initialized without bias. A lower triangular matrix (`tril`) of size `context_length` x `context_length` is registered as a buffer to implement causal masking, preventing the attention mechanism from attending to future tokens.

Single Head Attention by [Fareed Khan](#)

```

# --- Attention Head Class ---

class Head(nn.Module):
    """
    A single attention head.

    This module calculates attention scores and applies them to the values.
    It includes key, query, and value projections, and uses causal masking
    to prevent attending to future tokens.
    """
    def __init__(self, head_size, n_embed, context_length):
        super().__init__()
        self.key = nn.Linear(n_embed, head_size, bias=False) # Key projection
        self.query = nn.Linear(n_embed, head_size, bias=False) # Query projection

```

```

self.value = nn.Linear(n_embed, head_size, bias=False) # Value projection
# Lower triangular matrix for causal masking
self.register_buffer('tril', torch.tril(torch.ones(context_length, context_length)))

def forward(self, x):
    """
    Forward pass through the attention head.

    Args:
        x (torch.Tensor): Input tensor of shape (B, T, C).

    Returns:
        torch.Tensor: Output tensor after applying attention.
    """
    B, T, C = x.shape
    k = self.key(x) # (B, T, head_size)
    q = self.query(x) # (B, T, head_size)
    scale_factor = 1 / math.sqrt(C)
    # Calculate attention weights: (B, T, head_size) @ (B, head_size, T) -> (B, T, T)
    attn_weights = q @ k.transpose(-2, -1) * scale_factor
    # Apply causal masking
    attn_weights = attn_weights.masked_fill(self.tril[:T, :T] == 0, float('-inf'))
    attn_weights = F.softmax(attn_weights, dim=-1)
    v = self.value(x) # (B, T, head_size)
    # Apply attention weights to values
    out = attn_weights @ v # (B, T, T) @ (B, T, head_size) -> (B, T, head_size)
    return out

```

Our attention head class `__init__` method initializes linear layers for key, query, and value projections, each projecting the input embedding (`n_embed`) to `head_size`. A lower triangular matrix based on `context_length` is used for causal masking. The `forward` method calculates attention weights by scaling the dot product of the query and key, applies the causal mask, normalizes the weights using softmax, and computes the weighted sum of the values to produce the attention output.

Multi Head Attention

To capture diverse relationships within the input sequence, we are going to use the concept of multi-head attention. The `MultiHeadAttention` module manages multiple independent attention heads operating in parallel.

The key parameter here is `n_head`, which determines the number of parallel attention heads. The input embedding dimension (`n_embed`) and `context_length` are also necessary to instantiate the individual attention heads. Each head processes the input independently, projecting it into a lower-dimensional subspace of size `n_embed // n_head`. By having multiple heads, the model can attend to different aspects of the input simultaneously.

```
# --- Multi-Head Attention Class ---

class MultiHeadAttention(nn.Module):
    """
    Multi-Head Attention module.

    This module combines multiple attention heads in parallel. The outputs of each head
    are concatenated to form the final output.
    """
    def __init__(self, n_head, n_embed, context_length):
        super().__init__()
        self.heads = nn.ModuleList([Head(n_embed // n_head, n_embed, context_length) for _ in range(n_head)])

    def forward(self, x):
        """
        Forward pass through the multi-head attention.

        Args:
            x (torch.Tensor): Input tensor of shape (B, T, C).

        Returns:
            torch.Tensor: Output tensor after concatenating the outputs of all heads.
        """
        # Concatenate the output of each head along the last dimension (C)
        x = torch.cat([h(x) for h in self.heads], dim=-1)
        return x
```

Now that we have defined the `MultiHeadAttention` class, which combines multiple attention heads, the `__init__` method initializes a list of `Head` instances (a total of `n_head`), each with a `head_size` of `n_embed // n_head`. The `forward` method applies each attention head to the input `x` and concatenates their outputs along the last dimension, merging the information learned by each head.

Transformer Block

To create a billion-parameter model, we definitely need a deep architecture. For that, we need to code a transformer block and stack them. The key parameters of a block are `n_head`, `n_embed`, and `context_length`. Each block comprises a multi-head attention layer and a feed-forward network (MLP), with layer normalization applied before each and residual connections after each.

Layer normalization, parameterized by the embedding dimension `n_embed`, helps stabilize training. The multi-head attention mechanism, as described before, takes `n_head`, `n_embed`, and `context_length`. The MLP also utilizes the embedding dimension `n_embed`. These components work together to process the input and learn complex patterns.

```
# --- Transformer Block Class ---

class Block(nn.Module):
    """
    A single Transformer block.
```

This block consists of a multi-head attention layer followed by an MLP, with layer normalization and residual connections.

```
"""
def __init__(self, n_head, n_embed, context_length):
    super().__init__()
    self.ln1 = nn.LayerNorm(n_embed)
    self.attn = MultiHeadAttention(n_head, n_embed, context_length)
    self.ln2 = nn.LayerNorm(n_embed)
    self.mlp = MLP(n_embed)

def forward(self, x):
    """
    Forward pass through the Transformer block.

    Args:
        x (torch.Tensor): Input tensor.

    Returns:
        torch.Tensor: Output tensor after the block.
    """
    # Apply multi-head attention with residual connection
    x = x + self.attn(self.ln1(x))
    # Apply MLP with residual connection
    x = x + self.mlp(self.ln2(x))
    return x

def forward_embedding(self, x):
    """
    Forward pass focusing on the embedding and attention parts.

    Args:
        x (torch.Tensor): Input tensor.

    Returns:
        tuple: A tuple containing the output after MLP embedding and the residual.
    """
    res = x + self.attn(self.ln1(x))
    x = self.mlp.forward_embedding(self.ln2(res))
    return x, res
```

Our `Block` class represents a single transformer block. The `__init__` method initializes layer normalization layers (`ln1`, `ln2`), a `MultiHeadAttention` module, and an MLP module, all parameterized by `n_head`, `n_embed`, and `context_length`.

The `forward` method implements the block's forward pass, applying layer normalization and multi-head attention with a residual connection, followed by another layer normalization and the MLP, again with a residual connection. The `forward_embedding` method provides an alternative forward pass focused on the attention and initial MLP embedding stages.

The Final Model

So far, we have coded small components of the transformer model. Next, we integrate token and position embeddings with a series of transformer blocks to perform sequence-to-sequence tasks. To do that, we need to code several key parameters: `n_head`, `n_embed`, `context_length`, `vocab_size`, and `N_BLOCKS`.

`vocab_size` determines the size of the token embedding layer, mapping each token to a dense vector of size `n_embed`. The `context_length` parameter is important for the position embedding layer, which encodes the position of each token in the input sequence, also with dimension `n_embed`. The number of attention heads (`n_head`) and the number of blocks (`N_BLOCKS`) dictate the depth and complexity of the network.

These parameters collectively define the architecture and capacity of the transformer model, so let's code it.

Transformer Class by Fareed Khan

```
# --- Transformer Model Class ---

class Transformer(nn.Module):
    """
    The main Transformer model.

    This class combines token and position embeddings with a sequence of Transformer blocks
    and a final linear layer for language modeling.
    """
    def __init__(self, n_head, n_embed, context_length, vocab_size, N_BLOCKS):
        super().__init__()
        self.context_length = context_length
        self.N_BLOCKS = N_BLOCKS
        self.token_embed = nn.Embedding(vocab_size, n_embed)
        self.position_embed = nn.Embedding(context_length, n_embed)
        self.attn_blocks = nn.ModuleList([Block(n_head, n_embed, context_length) for _ in range(N_BLOCKS)])
        self.layer_norm = nn.LayerNorm(n_embed)
        self.lm_head = nn.Linear(n_embed, vocab_size)
        self.register_buffer('pos_idx', torch.arange(context_length))

    def _pre_attn_pass(self, idx):
        """
        Combines token and position embeddings.

        Args:
            idx (torch.Tensor): Input token indices.

        Returns:
            torch.Tensor: Sum of token and position embeddings.
        """
        B, T = idx.shape
        tok_embedding = self.token_embed(idx)
        pos_embedding = self.position_embed(self.pos_idx[:T])
        return tok_embedding + pos_embedding

    def forward(self, idx, targets=None):
        """
        Forward pass through the Transformer.

        Args:
            idx (torch.Tensor): Input token indices.
            targets (torch.Tensor, optional): Target token indices for loss calculation. Defaults to None.

        Returns:
            tuple: Logits and loss (if targets are provided).
        """
```

```

        """
        x = self._pre_attn_pass(idx)
        for block in self.attn_blocks:
            x = block(x)
        x = self.layer_norm(x)
        logits = self.lm_head(x)
        loss = None
        if targets is not None:
            B, T, C = logits.shape
            flat_logits = logits.view(B * T, C)
            targets = targets.view(B * T).long()
            loss = F.cross_entropy(flat_logits, targets)
        return logits, loss

    def forward_embedding(self, idx):
        """
        Forward pass focusing on the embedding and attention blocks.

        Args:
            idx (torch.Tensor): Input token indices.

        Returns:
            tuple: Output after attention blocks and the residual.
        """
        x = self._pre_attn_pass(idx)
        residual = x
        for block in self.attn_blocks:
            x, residual = block.forward_embedding(x)
        return x, residual

    def generate(self, idx, max_new_tokens):
        """
        Generates new tokens given a starting sequence.

        Args:
            idx (torch.Tensor): Initial sequence of token indices.
            max_new_tokens (int): Number of tokens to generate.

        Returns:
            torch.Tensor: The extended sequence of tokens.
        """
        for _ in range(max_new_tokens):
            idx_cond = idx[:, -self.context_length:]
            logits, _ = self(idx_cond)
            logits = logits[:, -1, :]
            probs = F.softmax(logits, dim=-1)
            idx_next = torch.multinomial(probs, num_samples=1)
            idx = torch.cat((idx, idx_next), dim=1)
        return idx

```

Our `Transformer` class `__init__` method initializes token and position embedding layers (`token_embed` , `position_embed`), a sequence of `Block` modules (`attn_blocks`), a final layer normalization layer (`layer_norm`), and a linear layer for language modeling (`lm_head`).

The `_pre_attn_pass` method combines token and position embeddings. The `forward` method processes the input sequence through the embedding layers and the series of transformer blocks, applies final layer normalization, and generates logits. It also calculates the loss if targets are provided. The `forward_embedding` method provides an intermediate forward pass up to the output of the attention blocks, and the `generate` method implements token generation.

Batch Processing

When we train a deep learning model on big data, we process it in batches due to GPU availability. So, let's create a `get_batch_iterator` function, taking the `data_path` to an HDF5 file, the desired `batch_size`, the `context_length` for each sequence, and the `device` to load the data onto.

The `batch_size` determines how many sequences are processed in parallel during training, while the `context_length` specifies the length of each input sequence. The `data_path` points to the location of the training data.

```

# --- Data Loading Utility ---

def get_batch_iterator(data_path, batch_size, context_length, device="gpu"):

```



```

"""
Creates an iterator for generating batches of data from an HDF5 file.

Args:
    data_path (str): Path to the HDF5 file containing tokenized data.
    batch_size (int): Number of sequences in each batch.
    context_length (int): Length of each sequence.
    device (str, optional): Device to load the data onto ('cpu' or 'cuda'). Defaults to 'cpu'.

Yields:
    tuple: A tuple containing input sequences (xb) and target sequences (yb).
"""
# Open the HDF5 file in read mode
with h5py.File(data_path, 'r') as hdf5_file:

    # Extract the dataset of tokenized sequences
    dataset = hdf5_file['tokens']

    # Get the total size of the dataset
    dataset_size = dataset.shape[0]

    # Calculate the number of examples (sequences) that can be made from the data
    n_examples = (dataset_size - 1) // context_length

    # Create an array of indices for examples and shuffle them for randomness
    example_idxs = np.arange(n_examples)
    np.random.shuffle(example_idxs)

    # Initialize epoch counter and example counter
    epochs = 0
    counter = 0

    while True:
        # Check if the current batch exceeds the number of available examples
        if counter + batch_size > n_examples:
            # Shuffle the indices again and reset the counter to 0
            np.random.shuffle(example_idxs)
            counter = 0
            print(f"Finished epoch {epochs}") # Print epoch number when an epoch finishes
            epochs += 1 # Increment the epoch counter

        # Select a batch of random indices to generate sequences
        random_indices = example_idxs[counter:counter+batch_size] * context_length

        # Retrieve sequences from the dataset based on the random indices
        random_samples = torch.tensor(np.array([dataset[idx:idx+context_length+1] for idx in random_indices]))

        # Separate the input sequences (xb) and target sequences (yb)
        xb = random_samples[:, :context_length].to(device) # Input sequence (first half of the random sample)
        yb = random_samples[:, 1:context_length+1].to(device) # Target sequence (second half of the random sample)

        # Increment the counter to move to the next batch
        counter += batch_size

        # Yield the input and target sequences as a tuple for the current batch
        yield xb, yb

```

Our `get_batch_iterator` function handles the loading and batching of training data. It takes `data_path`, `batch_size`, `context_length`, and `device` as input. The function opens the HDF5 file, shuffles the data, and then enters an infinite loop to generate batches. In each iteration, it selects a random subset of the data to form a batch of input sequences (`xb`) and their corresponding target sequences (`yb`).

Training Parameters

Now that we have coded our model, we need to define the training parameters, such as the number of heads, blocks, and more, along with the data path.

```

# --- Configuration ---

# Define vocabulary size and transformer configuration
VOCAB_SIZE = 50304 # Number of unique tokens in the vocabulary
CONTEXT_LENGTH = 512 # Maximum sequence length for the model
N_EMBED = 2048 # Dimension of the embedding space

```

```

N_HEAD = 16                # Number of attention heads in each transformer block
N_BLOCKS = 64              # Number of transformer blocks in the model

# Paths to training and development datasets
TRAIN_PATH = "data/train/pile_val.h5" # File path for the training dataset
DEV_PATH = "data/val/pile_val.h5"    # File path for the validation dataset

# Transformer training parameters
T_BATCH_SIZE = 32          # Number of samples per training batch
T_CONTEXT_LENGTH = 16      # Context length for training batches
T_TRAIN_STEPS = 200000     # Total number of training steps
T_EVAL_STEPS = 1000        # Frequency (in steps) to perform evaluation
T_EVAL_ITERS = 250         # Number of iterations to evaluate the model
T_LR_DECAY_STEP = 50000    # Step at which to decay the learning rate
T_LR = 5e-4                # Initial learning rate for training
T_LR_DECAYED = 5e-5        # Learning rate after decay
T_OUT_PATH = "models/transformer_B.pt" # Path to save the trained model

# Device configuration
DEVICE = 'cuda'

# Store all configurations in a dictionary for easy access and modification
default_config = {
    'vocab_size': VOCAB_SIZE,
    'context_length': CONTEXT_LENGTH,
    'n_embed': N_EMBED,
    'n_head': N_HEAD,
    'n_blocks': N_BLOCKS,
    'train_path': TRAIN_PATH,
    'dev_path': DEV_PATH,
    't_batch_size': T_BATCH_SIZE,
    't_context_length': T_CONTEXT_LENGTH,
    't_train_steps': T_TRAIN_STEPS,
    't_eval_steps': T_EVAL_STEPS,
    't_eval_iters': T_EVAL_ITERS,
    't_lr_decay_step': T_LR_DECAY_STEP,
    't_lr': T_LR,
    't_lr_decayed': T_LR_DECAYED,
    't_out_path': T_OUT_PATH,
    'device': DEVICE,
}

```

For most of the parameters, I have used the most common values and also stored them in a dictionary for easy access. Here, the parameters are for a billion-parameter model. If you want to train a model with millions of parameters, you can reduce the main parameters, which include `CONTEXT_LENGTH`, `N_EMBED`, `N_HEAD`, and `N_BLOCKS`. However, you can also run the million-parameter model script in my GitHub repository.

Training the Model

Let's initialize our transformer model and check its total number of parameters.

```

# --- Initialize the Model and Print Parameters ---

model = Transformer(
    n_head=config['n_head'],
    n_embed=config['n_embed'],
    context_length=config['context_length'],
    vocab_size=config['vocab_size'],
    N_BLOCKS=config['n_blocks']
).to(config['device'])

# Print the total number of parameters
total_params = sum(p.numel() for p in model.parameters())
print(f"Total number of parameters in the model: {total_params:,}")

#### OUTPUT ####
2,141,346,251

```

Now that we have 2 Billion parameter model, we need to define our Adam optimizer and loss tracking function, which will help us track the progress of our model throughout the training.

```
# --- Optimizer Setup and Loss Tracking ---

# Set up the AdamW optimizer with the specified learning rate.
optimizer = torch.optim.AdamW(model.parameters(), lr=config['t_lr'])

# List to track loss values during training.
losses = []

# Define a window size for averaging recent losses in the training loop.
AVG_WINDOW = 64

# Helper function to estimate the average loss for training and development data.
@torch.no_grad()
def estimate_loss(steps):
    """
    Evaluate the model on training and development datasets and calculate average loss.

    Args:
        steps (int): Number of steps to evaluate.

    Returns:
        dict: Dictionary containing average losses for 'train' and 'dev' splits.
    """
    out = {}
    model.eval() # Set the model to evaluation mode.

    for split in ['train', 'dev']:
        # Select the appropriate data path for the current split.
        data_path = config['train_path'] if split == 'train' else config['dev_path']

        # Create a batch iterator for evaluation.
        batch_iterator_eval = get_batch_iterator(
            data_path, config['t_batch_size'], config['t_context_length'], device=config['device']
        )

        # Initialize a tensor to track loss values for each evaluation step.
        losses_eval = torch.zeros(steps)
        for k in range(steps):
            try:
                # Fetch a batch and calculate the loss.
                xb, yb = next(batch_iterator_eval)
                _, loss = model(xb, yb)
                losses_eval[k] = loss.item()
            except StopIteration:
                # Handle the case where the data iterator ends early.
                print(f"Warning: Iterator for {split} ended early.")
                break

        # Compute the mean loss for the current split.
        out[split] = losses_eval[:k + 1].mean()

    model.train() # Restore the model to training mode.
    return out
```

We will now initialize our batch processing function and training loop, which will start our training.

```
# --- Training Loop ---

# Create a batch iterator for the training data.
batch_iterator = get_batch_iterator(
    config['train_path'],
    config['t_batch_size'],
    config['t_context_length'],
    device=config['device']
)

# Create a progress bar to monitor training progress.
pbar = tqdm(range(config['t_train_steps']))
for step in pbar:
```

```

try:
    # Fetch a batch of input and target data.
    xb, yb = next(batch_iterator)

    # Perform a forward pass and compute the loss.
    _, loss = model(xb, yb)

    # Record the loss for tracking.
    losses.append(loss.item())
    pbar.set_description(f"Train loss: {np.mean(losses[-AVG_WINDOW:]):.4f}")

    # Backpropagate the loss and update the model parameters.
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()

    # Periodically evaluate the model on training and development data.
    if step % config['t_eval_steps'] == 0:
        train_loss, dev_loss = estimate_loss(config['t_eval_iters']).values()
        print(f"Step: {step}, Train loss: {train_loss:.4f}, Dev loss: {dev_loss:.4f}")

    # Decay the learning rate at the specified step.
    if step == config['t_lr_decay_step']:
        print('Decaying learning rate')
        for g in optimizer.param_groups:
            g['lr'] = config['t_lr_decayed']
except StopIteration:
    # Handle the case where the training data iterator ends early.
    print("Training data iterator finished early.")
    break

```

Saving the Trained Model

Since our training loop has the ability to handle errors, in case the loop throws any error, it will save our partially trained model to avoid loss. Once the training is complete, we can save our trained model to use it later for inference.

```

# --- Save Model and Final Evaluation ---

# Perform a final evaluation of the model on training and development datasets.
train_loss, dev_loss = estimate_loss(200).values()

# Ensure unique model save path in case the file already exists.
modified_model_out_path = config['t_out_path']
save_tries = 0
while os.path.exists(modified_model_out_path):
    save_tries += 1
    model_out_name = os.path.splitext(config['t_out_path'])[0]
    modified_model_out_path = model_out_name + f"_{save_tries}" + ".pt"

# Save the model's state dictionary, optimizer state, and training metadata.
torch.save(
    {
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'losses': losses,
        'train_loss': train_loss,
        'dev_loss': dev_loss,
        'steps': len(losses),
    },
    modified_model_out_path
)
print(f"Saved model to {modified_model_out_path}")
print(f"Finished training. Train loss: {train_loss:.4f}, Dev loss: {dev_loss:.4f}")

```

The final training loss for the billion-parameter model is 0.2314, and the dev loss is 0.643.

Training Loss

When I plot the loss of both the million- and billion-parameter models, they look very different.

Training Loss Comparison

The billion-parameter model starts with a much higher loss and fluctuates a lot at the beginning. It goes down quickly at first, but then wobbles before becoming smoother. This shows that the bigger model has a harder time finding the right way to learn at the start. It might need more data and careful settings. When the learning rate is lowered (the red line), the loss goes down more steadily, showing that this helps it fine-tune.

The million-parameter model's loss goes down more easily from the start. It doesn't fluctuate as much as the bigger model. When the learning rate is lowered, it doesn't change the curve as much. This is likely because the smaller model is simpler to train and finds a good solution faster. The big difference shows how much harder it is to train very large models. They need different methods and maybe more time to learn well.

We now have our saved model. We can finally use it for inference and see how it generates text. 🤖

Generating Text

Let's create a function to generate text from our saved model, which takes the saved model path and the encoder as inputs and returns the generated text.

```
def generate_text(model_path, input_text, max_length=512, device="gpu"):
    """
    Generate text using a pre-trained model based on the given input text.

    Args:
    - model_path (str): Path to the model checkpoint.
    - device (torch.device): Device to load the model on (e.g., 'cpu' or 'cuda').
    - input_text (str): The input text to seed the generation.
    - max_length (int, optional): Maximum length of generated text. Defaults to 512.

    Returns:
    - str: The generated text.
    """

    # Load the model checkpoint
    checkpoint = torch.load(model_path)

    # Initialize the model (you should ensure that the Transformer class is defined elsewhere)
    model = Transformer().to(device)

    # Load the model's state dictionary
    model.load_state_dict(checkpoint['model_state_dict'])

    # Load the tokenizer for the GPT model (we use 'r50k_base' for GPT models)
    enc = tiktoken.get_encoding('r50k_base')

    # Encode the input text along with the end-of-text token
    input_ids = torch.tensor(
        enc.encode(input_text, allowed_special={'<|endoftext|>'}),
        dtype=torch.long
    )[None, :].to(device) # Add batch dimension and move to the specified device

    # Generate text with the model using the encoded input
```

```

with torch.no_grad():
    # Generate up to 'max_length' tokens of text
    generated_output = model.generate(input_ids, max_length)

    # Decode the generated tokens back into text
    generated_text = enc.decode(generated_output[0].tolist())

return generated_text

```

The transformer we defined earlier needs to be called here to load the architecture, and then we load the saved model as the state in that architecture.

Let's first observe what both the million and billion-parameter models generate without providing any input, and see what they generate randomly.

```

# Defining the file paths for the pre-trained models
Billion_model_path = 'models/transformer_B.pt' # Path to the Billion model
Million_model_path = 'models/transformer_M.pt' # Path to the Million model

# Using '<|endoftext|>' as input to the models (acts as a prompt that allows the models to generate text freely)
input_text = "<|endoftext|>"

# Call the function to generate text based on the input text using the Billion model
B_output = generate_text(Billion_model_path, input_text)

# Call the function to generate text based on the input text using the Million model
M_output = generate_text(Million_model_path, input_text)

# Print the output generated by both models
print(B_output) # Output from the Billion model
print(M_output) # Output from the Million model

```

Both LLMs are able to generate clear and accurate words when the context is short and simple. For example, in the million-parameter output, the phrase “**The villages were directly linked to cities in China**” makes sense and conveys a clear idea. It is easy to understand and logically connects the villages to the cities.

However, when the context becomes longer and more complex, the clarity begins to fade. In the billion-parameter output, sentences like “**There are two miles east coast from 1037 and 73 million refugees (hypotetus)**” and “**blacksmith, musician and boutique hospitality and inspire the strain delivered Canadians**” become harder to follow. The ideas seem disjointed, and the sentence structure doesn’t flow naturally. While the words used might still be correct, the overall meaning becomes confusing and unclear.

The positive point is that the 13+ million-parameter LLM also starts generating some kind of meaningful content with correct word spelling. For instance, when I use the subject input text, it starts generating an email for me. Although, obviously, broader text doesn’t provide meaningful results, take a look at the output:

```
# Input text
input_text "Subject: "

# Call the Million parameter Mod
m_output = generate_text(Million_model_path, input_text)
```

```
print(m_output) # Output from the Million model
```

Our million parameter model gives us the motivation that we can have a very narrow, goal-oriented LLM under 1B in size, while our 1B trained model shows us that the architecture needs to be coded in great depth with proper consideration. Otherwise, it won't improve training or performance compared to the million-parameter model. It will just overfit the data unless you have a deep architecture for the billion-sized model.

What's Next

I recommend that you create the 13+ million-parameter model and then start scaling it by adding the next 100 parameters, improving its ability to handle shorter contexts. It's up to you how many more parameters you want to train for specific tasks. Then, for the remaining parameters under 1B, try fine-tuning the model on domain-specific data, such as writing emails or essays, and see how it generates the text.

Wanna chat on something? [My LinkedIn](#)

Artificial Intelligence

Python

Data Science

AI

Machine Learning



Follow

Published in Level Up Coding

268K followers · Last published 17 hours ago

Coding tutorials and news. The developer homepage gitconnected.com && skilled.dev && levelup.dev



Follow

Written by Fareed Khan

54K followers · 0 following

I write on AI, <https://www.linkedin.com/in/fareed-khan-dev/>

Responses (28)



Erdogan T

What are your thoughts?



Lutfi Andriyanto
Jan 17



wow very amazing project



53

[Reply](#)



Klaus Belsner
Jan 16



Dear Fareed, nice post, but what type of GPU would you recommend (within the limits of "normal" enthusiasts?



23



1 reply

[Reply](#)



NTTP
Jan 17



Looks like fun! Do you know of any packaged software that lets you upload a corpus of text data and train? E.g. w/o all the programmy steps in Python. I tried a 100 billion post quantized model (trinary / ternary quantized I think) from another post... [more](#)



7

[Reply](#)

See all responses

More from the list: "LLMd"



Ramakrushna Mohapatra

Top 10 LLM & RAG Projects for Your AI...

★ · Aug 3



In Coding N... by Civil Lear...

Google's New LLM Runs on Just 0.5 GB RAM—...

★ · Aug 15



In Level Up C... by Fareed ...

Building the Entire RAG Ecosystem and...

★ · Aug 6



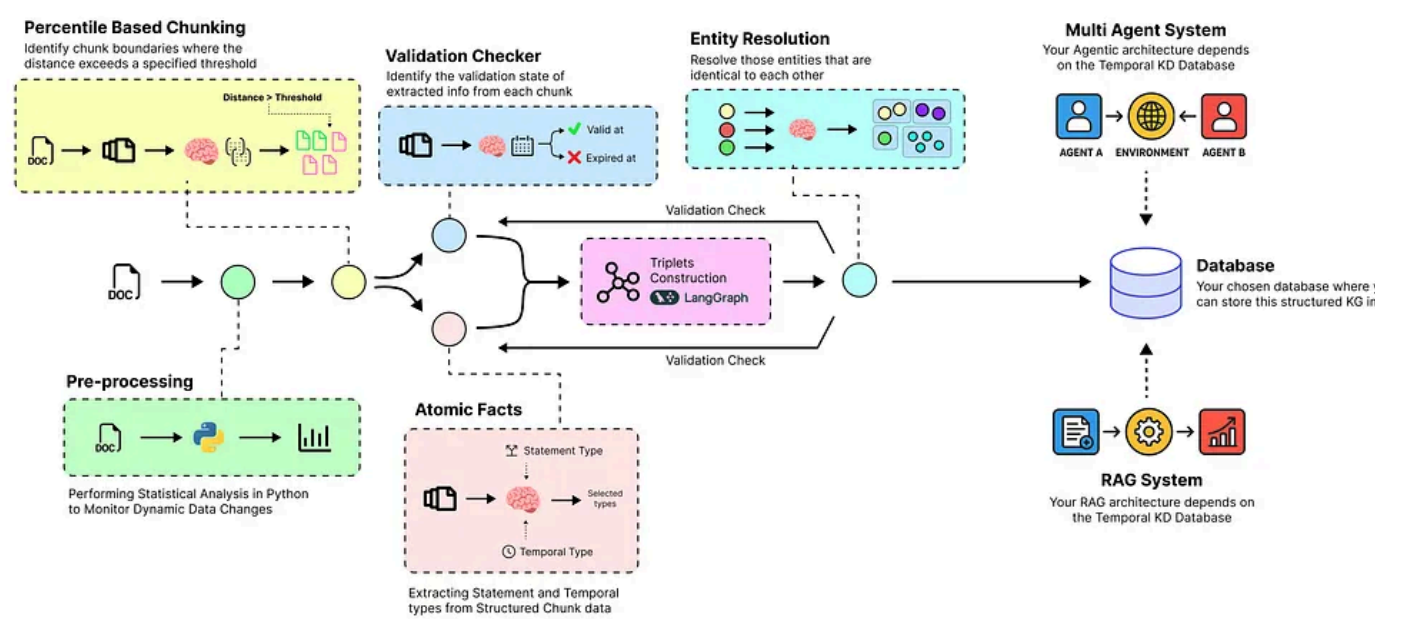
Sau Sheong

How to fine-tune an LLM

★ · Jun 18

[View list](#)

More from Fareed Khan and Level Up Coding



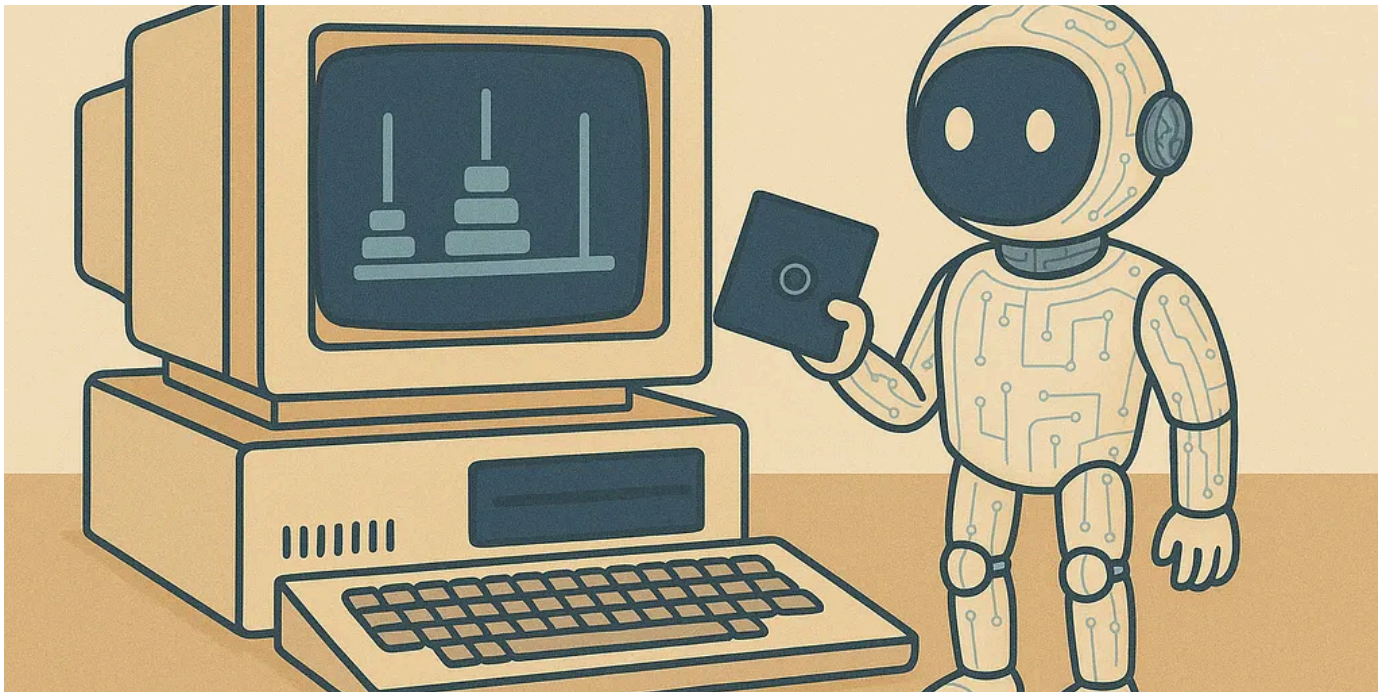
 In Level Up Coding by Fareed Khan

Building a Temporal AI Agent to Optimize Evolving Knowledge Bases in Modern RAG Systems

Atomic Facts, Validation, Entity Resolution, KG and more

★ Aug 15 🖱️ 790 💬 12

 ...



 In Level Up Coding by Marco Benedetti

Vibe Coding as a Coding Veteran

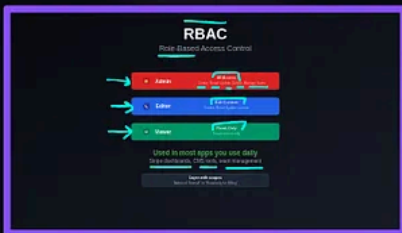
From 8-bit Assembly to English-as-Code

Aug 2  405  10

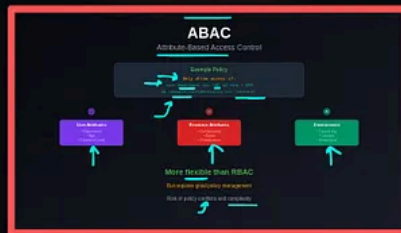


Authorization 101

RBAC



ABAC



ACL



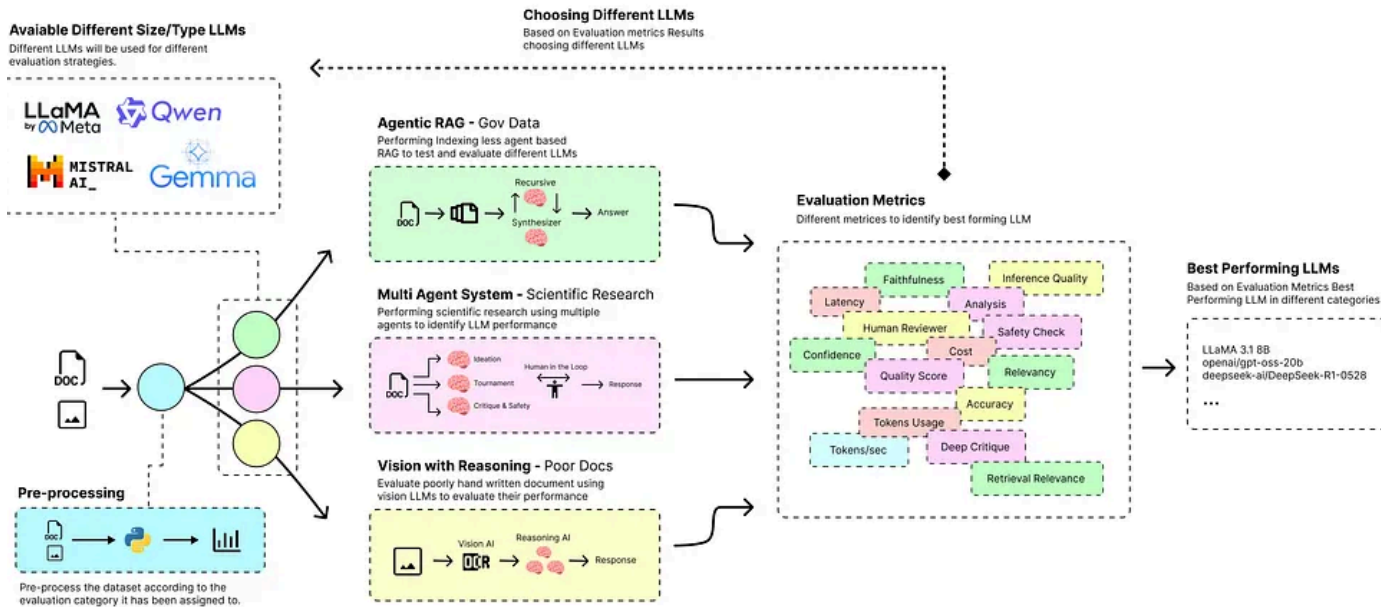
 In Level Up Coding by Hayk Simonyan

Authorization Explained: When to Use RBAC, ABAC, ACL & More

Learn how real-world systems control what users can do using authorization models like RBAC, ABAC, and ACL.

Aug 13  429  8





 In Level Up Coding by Fareed Khan

Building Three Pipelines to Select the Right LLMs for RAG, Multi-Agent Systems, and Vision

Agentic RAG, Multi-Agent and Vision with Reasoning

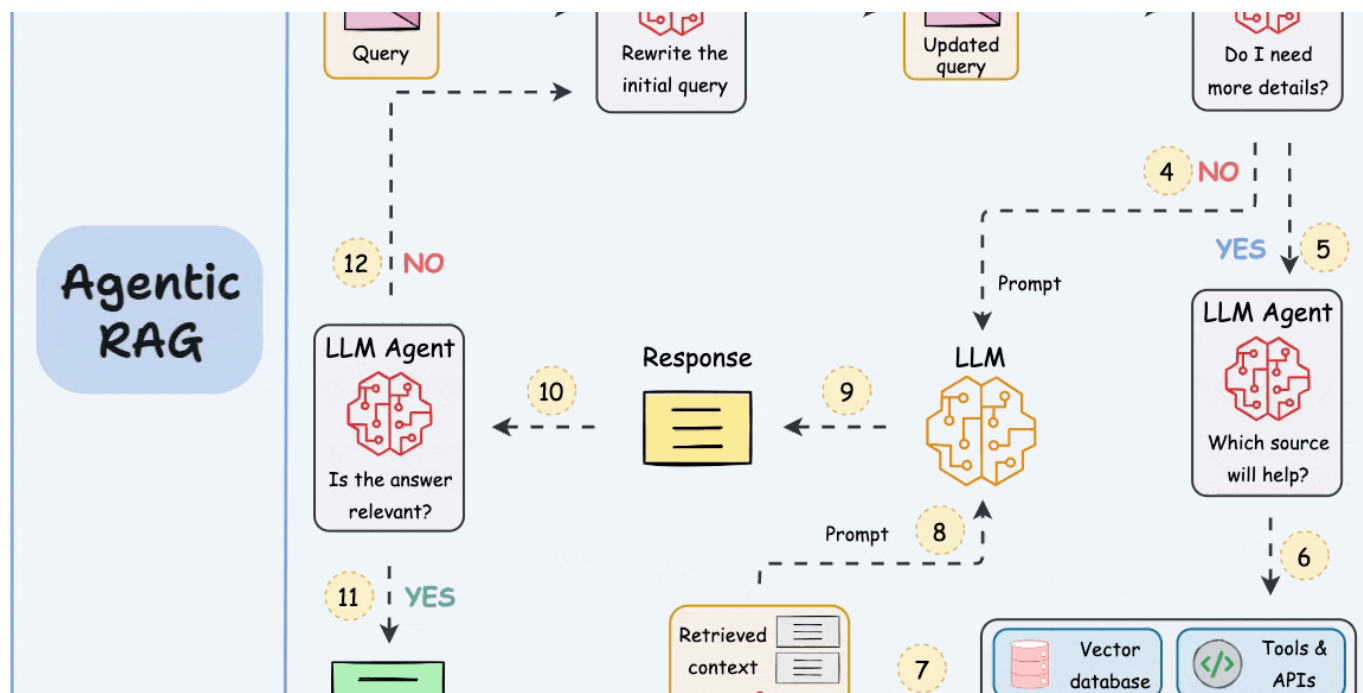
★ Aug 22 🖱 826 💬 11

🔖 ...

See all from Fareed Khan

See all from Level Up Coding

Recommended from Medium

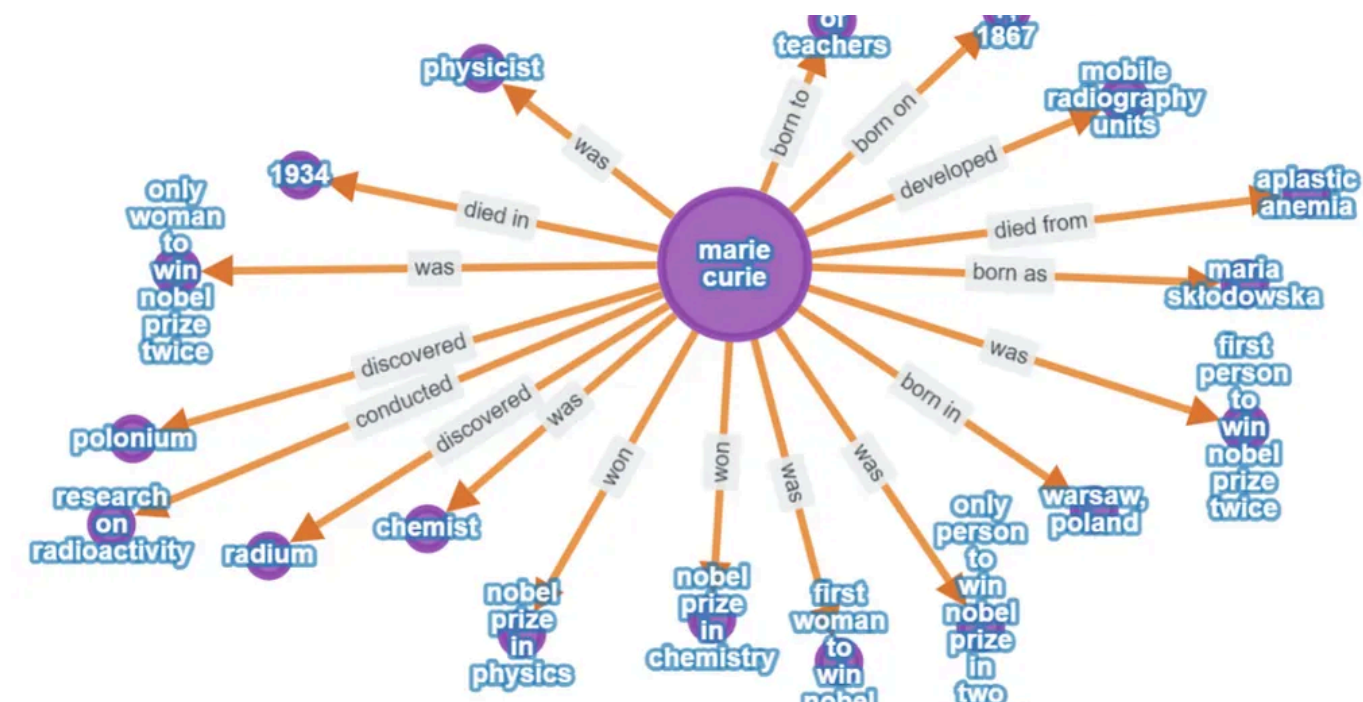


AI In Artificial Intelligence in Plain English by Piyush Agnihotri

Building Agentic RAG with LangGraph: Mastering Adaptive RAG for Production

Build intelligent RAG systems that know when to retrieve documents, search the web, or generate responses directly

★ Jul 20 📖 1.7K 💬 27



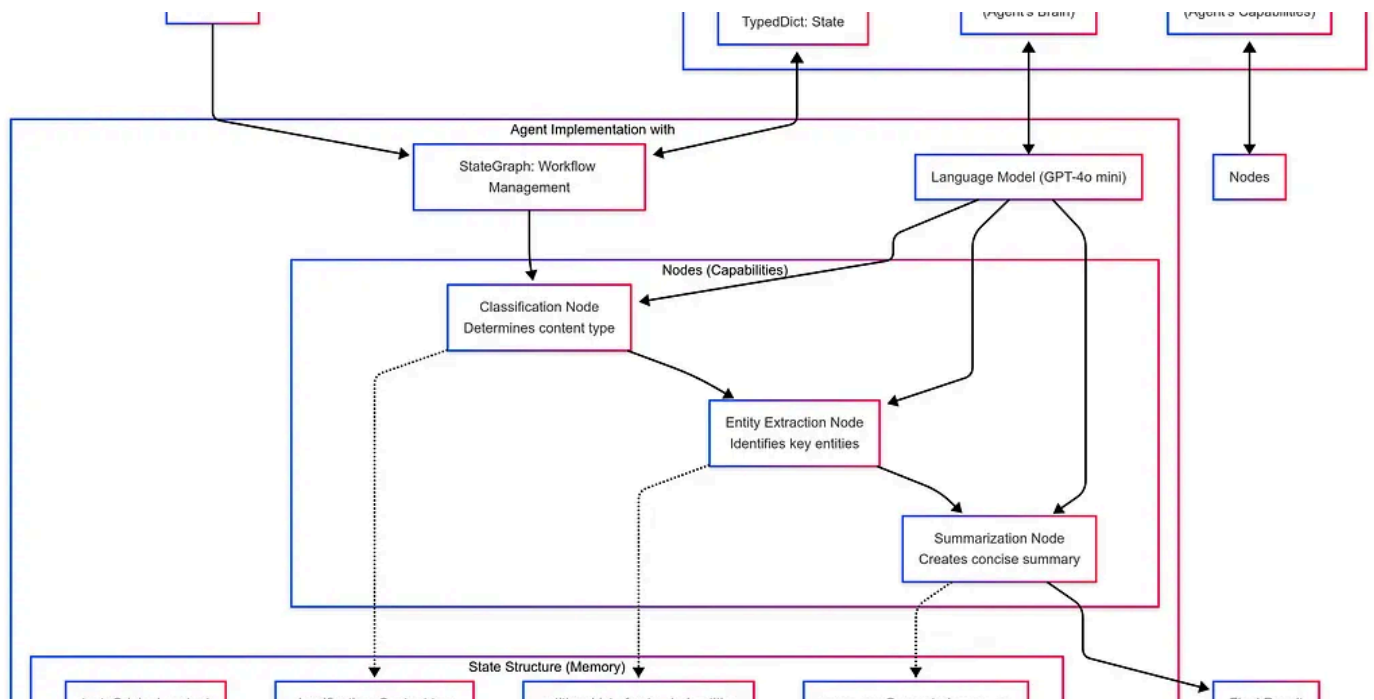
∞ In Level Up Coding by Fareed Khan

Converting Unstructured Data into a Knowledge Graph Using an End-to-End Pipeline

Step by Step guide

★ Apr 17 📖 2.4K 💬 46





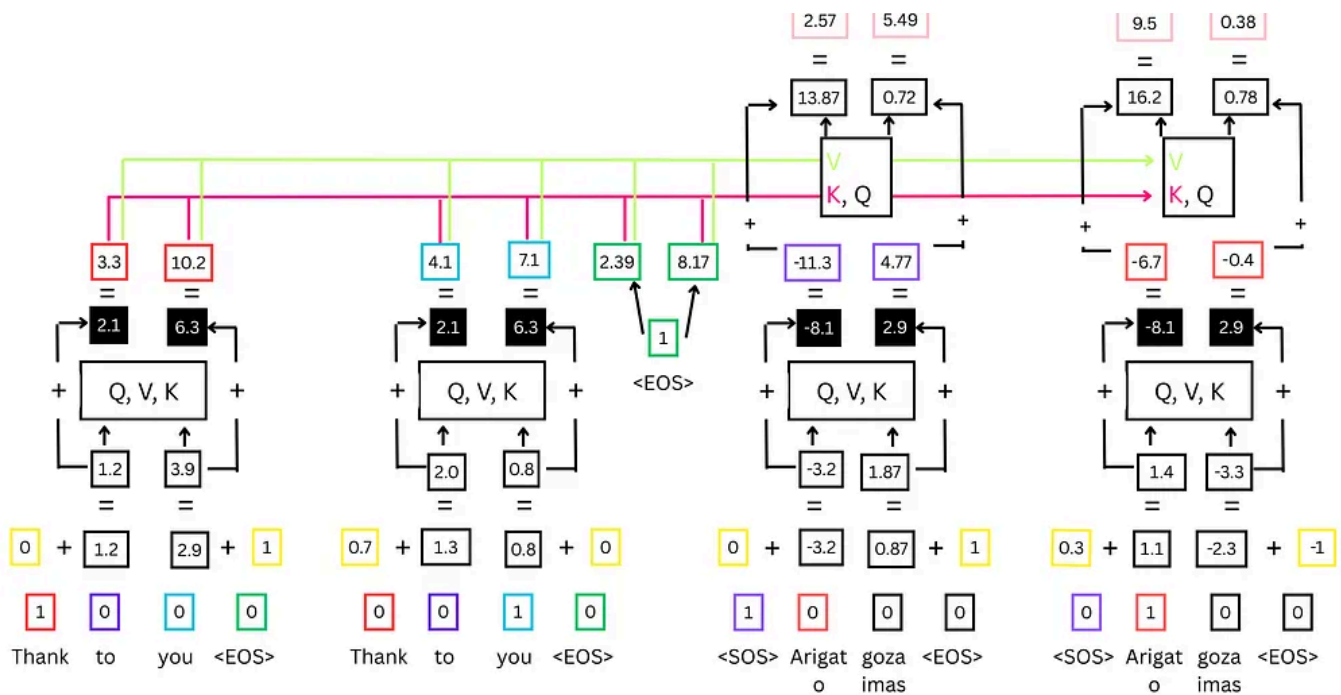
DSC In Data Science Collective by Paolo Perrone

The Complete Guide to Building Your First AI Agent with LangGraph. (It's Easier Than You Think)

Three months into building my first commercial AI agent, everything collapsed during the client demo.

★ Mar 11 4K 87

🔖 ...



Joshua Anang

The math and logic behind ChatGPT. This paper is all you need.

I'm Joshua Anang and I was 17 years old (last year) when I built my own version of ChatGPT. I'm writing this paper to explain the math...

May 7 1.98K 33

🔖 ...

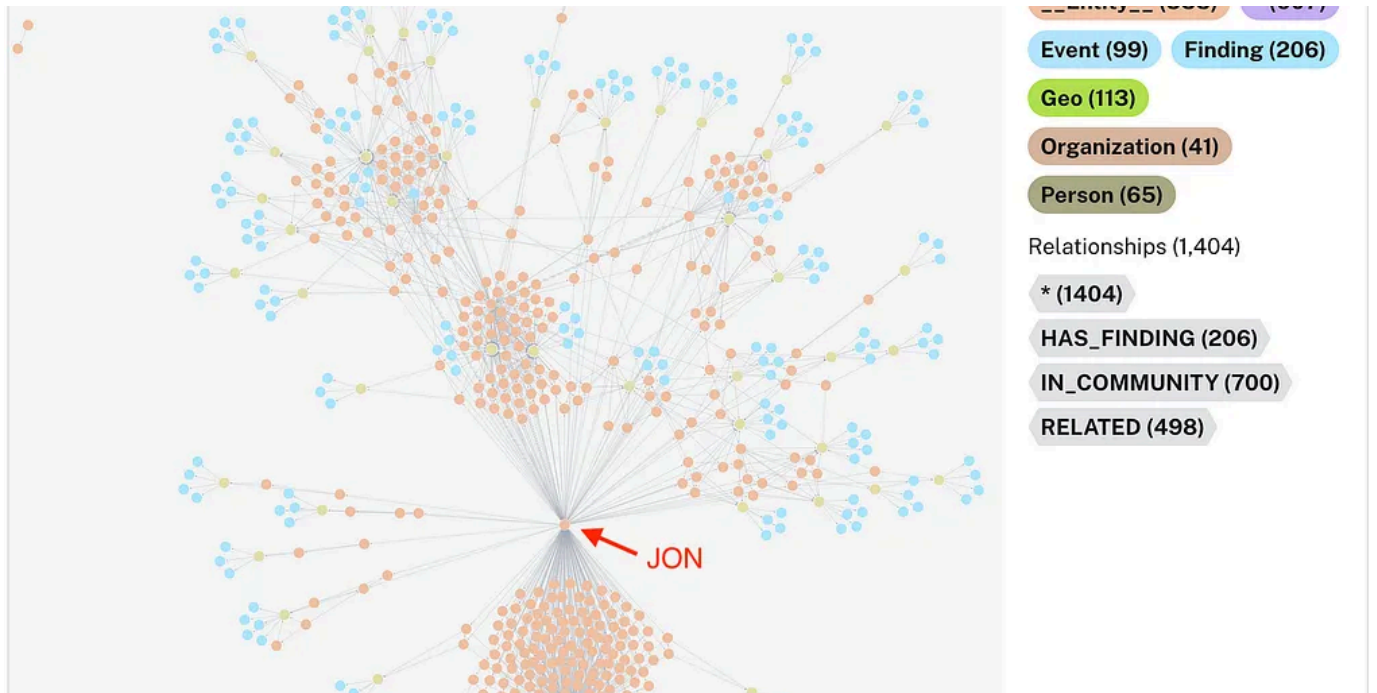


py In Python in Plain English by Mayuresh K

Architecture of AI-Driven Systems: What Every Technical Architect Should Know

A Step-by-Step Guide to Integrating External AI Services into Commercial Systems with Fallback Strategies

Aug 6 221 7



AI In Towards AI by Mariana Avelino

How GraphRAG Works Step-by-Step

Perhaps you've come across the paper From Local to Global: A GraphRAG Approach to Query-Focused Summarization, which is Microsoft...

May 6 1.1K 9



See more recommendations